LLM 自动算法设计

从指南纠错到独立复现

概念推导·视频与源码·实验规范·学习安排

24 周
主线学习计划

82 条
来源与资源记录

29 项
通过的教学测试

本手册围绕老师提供的入门指南，先修正概念、文献与代码问题，再以装箱、TSP 和 CVRP 连接经典算法、LLM 程序搜索及研究实验。目标是能够独立解释方法、运行小实验、识别证据边界，并设计可复核的研究。

审读对象：《LLM 自动算法设计入门指南 (1).docx》

检索核验截止：2026 年 9 月 19 日

版本：1.0　Word 与 PDF 共享正文；PDF 由 XeLaTeX 编译

目录

1 阅读方法与交付边界 1

2 原指南逐节审读：保留主线，重排依赖 1

3 必须先修正的事实、引文与代码问题 2

4 建立完整的概念模型 5

5 数学与算法：沿着三个可计算的例子理解 6

6 论文精读：把每篇读成一个可复现的实验 8

7 视频、课程与社区：按问题观看，而不是按播放量收藏 10

8 源码阅读与资源选择：先找到评估器，再找生成器 12

9 端到端最小系统：把可以信任的部分先建好 15

10 研究级实验协议：让改进经得住重新运行 17

11 24 周主线与前 14 天安排 19

12 研究候选与农业规划迁移 22

13 从学习产物到可审查的论文 23

14 附录：概念自测与参考答案 24

15 参考文献与资源登记 27

1 阅读方法与交付边界

这份手册以老师提供的《LLM 自动算法设计入门指南 (1).docx》为审读对象，目标不是把里面的名词都背下来，而是让你能够独立回答四个问题：**要设计什么算法；模型究竟改变了什么；改进由谁验证；证据能支持多大范围的结论**。本册同时提供学习顺序、公开视频与源码入口、实验设计、进度安排和可以直接运行的纠错练习。

原指南的研究方向值得认真学习，但它不是一份可以逐条照抄执行的实验协议。正文中存在文献标识符错配、框架身份混淆、过时的模型推荐和不可直接运行的示例。这里分别处理“方向是否有价值”“资料是否准确”“代码是否可用”，不让任何一项代替另外两项。

证据口径。检索核验截至 2026 年 9 月 19 日。原指南依据该文件可检索的正文逐节审读；原始 Word 字节导出未成功，因此不声称审查了嵌入对象、批注、修订历史或原始排版。外部资源以论文作者、出版机构、会议、大学课程和项目官方仓库为主；社区内容用于定位资源和解决工程问题，不替代一手证据。这是一份有检索记录的研究型学习设计，不是宣称穷尽全部平台的系统综述。

执行口径。本次实际执行了附带的遗传算法、微型 TSP、在线装箱及验证器练习，29 项测试通过。没有在本次会话中完成 EoH、FunSearch、ReEvo 或大模型微调的整套论文复现；它们在本册中分别标为“已核验资料”“计划实验”。公开视频核验的是来源、主题与入口，不声称全部完整观看，也不承诺每个校园网和手机客户端均可播放。

时间口径。主线按 24 周、每周 14 小时设计，共 336 小时；不足的编程或数学基础另加 0—8 周，论文研究另加 8—12 周。这里的小时是建议投入，不是课程视频时长，也不是保证掌握知识的工期。先通过能力检查，再决定是否压缩或补课。

论文口径。“顶刊水准”在这里落实为可追溯引用、可检验假设、强且公平的基线、独立评估、预算核算、统计不确定性、明确的失败边界。漂亮排版、模型数量、代码体量和结果最好的一次，都不能替代这些要求。[P22]

1.1 先走哪一条阅读路线

第一次通读先看第 2—4 章，把指南纠错和核心问题看清，再看第 10 章实验协议与第 11 章时间安排。准备动手时读第 7—9 章；每完成一个练习再回看论文精读卡。第 12 章的研究候选不应在第一周就变成已经确定的“创新点”。所有资源编号均可点击；完整入口、核验状态与备注另见附录和随包的 CSV/JSON 目录。

2 原指南逐节审读：保留主线，重排依赖

2.1 原指南的八个部分，分别需要学到什么程度

原指南部分	应保留的学习目标	本册的调整与验收
1 引言	理解自动算法设计，而非只会调用聊天模型	能区分生成一个解、生成一个算法、调参数、选算法
2 基础知识	EA、NAS、程序合成、数学、Python、LLM	分成必须会用、必须理解、方向确定后深入三层
3 学习步骤	从基础到复现再到独立研究	用能力关卡替代“刷满 200 题”和每日 7—8 小时

原指南部分	应保留的学习目标	本册的调整与验收
4 代码与 GitHub	阅读真实项目并接通评估闭环	核对项目身份、真实 API、版本与可执行范围
5 关键论文	建立概念谱系、理解方法与证据	修正错引；把经典、平台、模型论文与近期预印本分开
6 实践项目	从小实验走到可复现实验	保留六类目标，不同时启动六个大工程
7 学习路线图	形成连续的产出与复盘	每周一个可检查成果，失败则补前置条件
8 链接汇总	能持续找到可信资源	建资源登记表，不以链接数或收藏量表示掌握程度

原指南没有把应用限定为农业、CVRP 或覆盖规划。本册后面的农业任务规划是结合你的研究语境给出的迁移建议，不冒充原指南的硬性规定。自动算法设计应是学习主线；TSP 与装箱先充当容易检查的教学任务，最终研究任务等完成基础复现后再选。

2.2 三种学习深度，不把所有前置课都当成门槛

必须亲手实现。输入检查、随机种子与实验记录、至少一个图搜索算法、TSP 路线代价与可行性验证、最近邻与 2-opt、First Fit/Best Fit、一个最小遗传算法、数据分组划分、简单统计汇总。这里的要求是离开原教程仍能实现，并能设计反例。

必须解释并做小实验。Attention 的矩阵形状与因果掩码，预训练/监督微调/强化学习的区别，程序合成的规格与验证，GA/PSO/DE 的更新对象，NAS 的搜索空间与评估代价，RL 的状态/动作/奖励，模仿学习的分布偏移。掌握这些，不等于需要先训练一个大型语言模型。

确定方向后再深入。大模型权重微调、复杂 NAS、多目标选择器、完整求解器自动生成、跨任务迁移、对抗式实例生成、多机大规模实验。它们不是“越多越好”的模块，而是针对具体瓶颈的手段。

2.3 原指南六类项目如何处理

GA/PSO 项目保留为评分和预算意识训练；代码模型生成排序函数保留为接口与正确性练习，但不绑定某个历史模型；EoH 复现是主线项目，不把微调列为先决条件；NAS 保留一个小型完整实验作为概念分支；综合系统只围绕一个经过验证的评估器搭建；研究项目最后才讨论创新。这既覆盖指南，也避免一年都在补课或安装框架。

3 必须先修正的事实、引文与代码问题

3.1 分类与论文身份

四角色分类被改写了。指南把 LLM 角色写成 Optimizer、Programmer、Evolver、Designer，并归于 Liu 等综述；本次核验的综述 v5 使用的是 optimizers、predictors、extractors、designers。你可以为了教学自定义“程序员”“进化算子”等角色，但必须声明这是自己的功能分类，不能当作原文术语。进一步说，“EoH 用了进化”不等于模型必然属于某一个互斥的单一类别。[P01]

FunSearch、AlphaTensor 和 AlphaEvolve 是三件不同的事。 FunSearch 用语言模型提出程序、通过评估和程序搜索改善数学构造及在线装箱启发式；AlphaTensor 以强化学习搜索矩阵

乘法算法；AlphaEvolve 是更一般的进化式代码改进系统。不能把 AlphaTensor 的标题、作者、年份和矩阵乘法贡献贴到 FunSearch 名下。[P03] [P06] [P25]

综述、算法论文、平台论文、模型论文不能相互替代。EoH 论文回答的是怎样搜索启发式；LLM4AD 回答的是怎样组织方法、任务和模型接口；CodeGen/StarCoder/CodeLlama 回答的是代码模型的训练与能力；HumanEval 主要测功能正确性。把它们统一排成“最新 SOTA 十篇”，会掩盖它们测量的对象完全不同。[P02] [P05] [P13] [P19] [P20] [C18]

3.2 文献标识符核验表

下表核对的是指南给出的“题名—链接”对应关系。链接能打开不代表引文正确；找不到某个条目也不等于证明世界上没有同名研究。

指南条目或标识符	核验结果	阅读与引用处理
自动算法设计综述	应使用 A Systematic Survey on Large Language Models for Algorithm Design，2410.14716	固定 v5，按其实际分类阅读 [P01]
代码 LLM 综述：2305.08857	实为 COWPEA，一篇选举方法论文 [A01]	替换为 Jiang 等代码生成综述 [P16]
程序合成综述：2308.12345	实为早期引力修正研究 [A02]	不保留错引；先学程序合成课程，再读 Austin 等研究论文 [V18] [P17]
NAS 综述：1908.09791	实为 Once-for-All，并非 Elsken 综述 [A03]	综述改为 1808.05377 [P15]
EA 与 DL 综述：2301.02345	实为手征 U(1) 规范理论中的轴子研究 [A04]	原条目撤出必读；不强配一个未经核验的替代标题
FunSearch：2205.00527	实为 Schmidt 问题的有限类比 [A05]；所写标题又属于 AlphaTensor	分别引用 FunSearch 与 AlphaTensor [P03] [P25]
AlphaEvolve：2501.12345	实为 The doubly librating Plutinos [A06]	改为 2506.13131 [P06]
LLM4AD：2502.23456	本次返回 404 [A07]	改为 2412.17287 [P05]
DARTS：1806.09055	题名匹配	保留，按 NAS 分支阅读 [P14]
AlphaCode：2203.07814	题名匹配	保留为代码合成背景，不当作优化启发式评估 [P12]
CodeGen：2203.13474	题名匹配	保留为历史模型论文 [P13]
EoH：2401.02051	题名匹配	主线精读；核对真正的五种提示策略 [P02]
CodeLlama / StarCoder / WizardCoder	2308.12950 / 2305.06161 / 2306.08568 对应论文存在	保留历史脉络，不直接称 2026 最新推荐 [P19] [P20] [P21]
“Self-Evolving Code Generation”等五个泛化标题	指南缺少足够的作者、会场、标识符信息	暂不计为五篇已识别的必读论文

指南把 WizardCoder 直接标为 “Microsoft”，不足以作为规范作者信息；按论文作者署名引用即可。类似 “支持商业使用”“最新 70B” 这样的判断，也不能由旧教程一句话替代当前模型卡、许可证与发布说明。本册不依据这些表述要求你下载历史大模型。[P21]

指南中的 “DSP（Dataflow Synthesis Planner）” 没有给出可追溯的作者、论文或仓库；本次精确词组检索未定位到可确认的对应项目。因此暂列身份未确认，不能根据缩写擅自补成另一个 DSP/DSPy 项目。

3.3 三个仓库，不要再装错

LLM4AlgorithmDesign 是综述配套文献集合；EoH 是方法实现；LLM4AD 是包含多种方法与任务的通用平台。指南给 LLM4AlgorithmDesign 写出的目录树和`examples/run_eoh.py` 安装命令，不能当作已核实的仓库文件。平台身份应从官方 README 与实际源码确定。[C01] [C02] [C03]

更重要的是，**真实仓库也会换 API**。本次读取 EoH README 时，主分支已使用 `LLMConfig / EoH / BaseProblem`；旧的 `Paras / eoh.EVOL` 对应旧接口，README 指向 v0.1。LLM4AD 则有自己的 `HttpsApi`、`OBPEvaluation` 和 `llm4ad.method.eoh` 接口。不要把两个项目的代码段拼在一起运行。当前主分支能用，也不等于它与 2024 论文设置相同。[C02] [C03]

3.4 两段示例代码的实质问题

遗传算法：先繁殖后评分的顺序错误。原示例给旧种群计算 fitness，然后创建 fitness 为 None 的子代，用子代替换种群，随即对 None 取 max 并格式化。这里不是 “效果可能不好”，而是执行顺序会触发错误。附带练习独立复现了这个操作顺序，实际得到 TypeError；没有冒称执行了无法导出的完整原文件。

方向与边界也需要明确。原示例用 max 选父代；若直接把 Sphere 函数作为目标，则它在最大化本应最小化的值。只有明确取负适应度或更换选择方向才正确。单点交叉在维度为 1 时没有合法切点；非有限返回值、空输入、种群规模和停止条件也没有完整处理。精英保留不是 GA 的必要定义，但若你承诺 “历史最好值单调不变差”，就必须显式保留并测试这个性质。[C12]

代码模型封装：生成文本不等于得到可执行候选。原示例把提示词和新生成内容一起解码，使用总长度限制而不是明确的新 token 预算，未完整处理模型设备、依赖、截断、代码提取、接口校验和执行隔离。纠正方向是：固定输入输出协议、保存原始响应、只提取合法候选、检查签名、独立评估；不是把生成结果交给 `exec` 就算完成。FP16、设备映射和依赖是否合适取决于具体模型与环境，不能简单判定为所有环境都错。[V12] [C18]

3.5 EoH 五种策略的正确理解

原论文把进化提示分成探索 E1/E2 和修改 M1/M2/M3：E1 倾向产生与父代不同的启发式；E2 提取父代共性并据此发展新启发式；M1 修改一个启发式；M2 调整其参数；M3 简化冗余部分。初始化另计，不能把五种策略随意改写成 “交叉、变异、反思、学习、初始化”。这些是启发式操作的语义，不是保证成功的定理。[P02]

4 建立完整的概念模型

4.1 实例、解、算法、算法设计器：四层对象

设 x 是一份问题实例，y 是一份候选解，h 是一个求解算法，M 是算法设计过程。普通求解计算 $y = h(x)$；自动算法设计计算 $h = M(D, B)$，其中 D 是可用于设计的实例集合，B 是设计预算。它们的输入、输出和泛化要求都不同。

以 TSP 为例，四个坐标点是实例，“0→1→2→3→0” 是解，“每次选最近未访问点” 是算法，“生成并筛选多个选点规则” 是算法设计器。让 LLM 为一张图输出一条路线，不等于它已经设计出可迁移到其他图的算法；让 LLM 照着说明实现最近邻，也不自动成为新算法。[P01] [P02]

易混概念	改变的对象	必须回答的问题
算法选择	从既有算法集合中选一个	特征获取和选择的代价是否计入?
算法配置	固定算法的参数	参数搜索是否比随机搜索更有效?
算法生成	规则、表达式、代码或结构	生成内容是不是能在新实例上反复使用?
神经求解器	模型权重与策略	学到的是直接构造解，还是帮助另一个求解器?
超启发式	选择或生成启发式的高层方法	决策层在哪，底层算子是否固定?
程序合成	满足规格的程序	规格是自然语言、测试用例，还是形式化逻辑?

这些范围可以重叠，但实验必须说清楚重叠在哪里。模型既提议参数又改写代码，就不能把其收益全部解释成 “选算法更聪明”。[P01] [P17] [V22]

4.2 双层优化：外层设计算法，内层执行算法

对于最小化任务，可以把设计目标写成：

$$h^\star \in \arg\min_{h \in \mathcal{H}} \mathbb{E}_{x \sim \mathcal{D}} \big[\ell(h, x; b) \big], \qquad \mathrm{Cost}_{\mathrm{design}} \le B.$$

这里 b 是每个求解算法执行一个实例时的预算，B 是搜索算法本身的预算。前者可以是固定 CPU 时间或函数评价次数，后者包括模型调用、候选评估、修复和选择开销。用五小时设计出一个启发式，再让它处理一万个实例，与每个实例都请求 LLM 重新求解，是不同的部署模式。

这个表达式并不意味着我们知道真实分布 $\mathcal{D}$，也不保证找到全局最优。实践中只得到有限数据上的估计值。需要独立验证集选择候选，再用封存测试集估计泛化。代码越自由、搜索越久，越有机会利用评估器漏洞或适应某一批样本；“搜索更多” 因此同时增加能力和过拟合风险。

4.3 从评分函数开始，不必直接生成整套求解器

一个最小的在线装箱接口可以写成：`score(item, remaining_capacities) -> scores`。固定外层程序负责剔除装不下的箱子、处理新箱、并列规则和更新状态；候选函数只负责给合法选择排序。这样，方法差异集中在一个可以测量的部件上。

随后再逐步扩展到“局部搜索的惩罚函数”“算子选择规则”“多个协作函数”“完整求解器”。表达能力越大，验证、调试和归因越困难。当前 EoH 仓库已出现函数、多函数和类等不同设计范围，说明单函数不是领域上限；但它仍然是最合适的第一份因果清晰的教学实验之一。[C02]

4.4 正确性、可行性、质量、安全性必须分开

“代码可以导入”只排除了部分语法错误；“返回一个列表”不保证它是合法路线；“路线合法”不保证代价算对；“代价较低”不保证没有读测试答案；“在子进程运行”也不保证安全。验证至少分为接口检查、结果可行性、独立目标重算和运行隔离四层。[C18]

用 HumanEval 等任务练习代码生成很有帮助，但优化实验还要报告路线代价、求解时间、失败率和分布外性能。pass@k 测的是采样得到正确程序的机会，不能替代启发式的优化质量，更不能忽略 k 次生成的预算。[C18]

5 数学与算法：沿着三个可计算的例子理解

5.1 在线装箱：最小的“算法设计”练习

箱子容量为 1，物品按 0.6、0.6、0.4、0.4 依次到达。Best Fit 选择放入后剩余容量最小的可行箱子；Next Fit 只尝试当前箱子。前者最终用 2 个箱子，后者用 3 个。附带代码计算并验证了这个差异。重要的不是记住谁赢，而是知道在线算法此刻不能读取未来物品；把未来物品喂给评分函数，就悄悄改变了问题。

令剩余容量为 r_j、当前物品为 a，Best Fit 可以在 $r_j \geq a$ 的合法箱子集合里最小化 $r_j - a$。若评估器采取最大分数选择，可令 $s_j = -(r_j - a)$。最大化评分与最小化实际目标不矛盾，前提是符号、掩码与并列规则都写清楚。

若以 $LB = \lceil \sum_i a_i / C \rceil$ 作为简单体积下界，使用箱数 N_h 可报告 $(N_h - LB)/LB$。必须称“相对该下界的差距”，不能称“最优差距”，因为这个下界未必紧。空实例需单独定义，不通过随意加一个很小的数掩盖分母为零。

第一轮实验只比较 Next Fit、First Fit、Best Fit 与同接口候选；保留原始物品顺序、容量、分布和随机种子。后续再增加接近半箱的困难物品、容量尺度变化和未见分布。它的价值在于评估器便宜、可行性好检查，不代表它天然最适合农业研究。

5.2 TSP：精确小实例与规模化启发式

对 n 个节点及代价矩阵 D，一条合法巡回访问每个节点恰好一次并回到起点：

$$L(\pi) = \sum_{i=0}^{n-1} D_{\pi_i, \pi_{(i+1) \bmod n}}.$$

四个单位正方形顶点按周边访问，长度为 4；选择一条含两条对角线的交叉路线，长度为 $2 + 2\sqrt{2} \approx 4.828427$。这里最优值可用枚举核对；一般大实例则不能把启发式找到的最好值自动当成已证明的最优值。

你应实现：固定起点的穷举小实例求解器、最近邻、2-opt 和独立路线验证器。穷举仅用于 $n \leq 9$ 的教学检查，不能拿它当大规模基线。对于对称 TSP，2-opt 可用边替换快速计算增量；对于非对

称代价，片段反转会改变内部边方向，照抄对称公式可能算错。因此先全量重算验证，再优化增量更新。

比例缩放、节点重编号和欧氏平移旋转是有用的测试。欧氏坐标旋转不会改变距离矩阵，但按坐标轴硬编码的规则可能变；存在并列距离时，不应强迫算法返回完全相同的节点序列，而应检查代价、可行性和事先声明的并列策略。

5.3 CVRP：从节点顺序走向多机器人任务分配

CVRP 给每个客户需求 q_i，每条车路线容量不超过 Q，从仓库出发并返回。不同车辆之间既要分配任务，也要安排顺序。因此“把 TSP 多跑几遍”不是一般正确的 CVRP 建模方法。容量、车辆数量是否固定、是否允许拆分需求、是否有时间窗，都要在问题定义中写明。[C13] [C14] [C15]

为教学目的，先做同质车辆、单仓库、确定性非负代价、不可拆分需求、无时间窗的 CVRP。用成熟求解器建立强基线，再考虑自己的候选。引入农业属性时，不要立即叠加所有约束；先回答某个约束是否真的改变可行域、代价或最优决策，再决定是否加入。

一个可复核的微型例子：容量 $Q = 10$，三个客户需求分别为 6、6、4，则总需求给出的车辆数下界为 2，但哪些点同车仍由路线代价决定。若一个客户需求为 11 且禁止拆分，则实例本身不可行；这是数据/模型层问题，不能被算法返回空路线掩盖。

5.4 EA、PSO、DE：看清“被更新的东西”

GA 可更新数值向量、排列、表达式或程序；交叉和变异必须尊重表示。普通实数交叉不能直接保证生成合法 TSP 排列。PSO 通常以位置和速度更新候选数值向量；DE 用若干个体的差分构造试验向量。它们不是因为名字不同就应全部塞进 LLM 框架。

入门时可用同一个 Sphere 目标 $f(z) = \sum_j z_j^2$ 比较 GA、随机搜索和一个 DE 实现，但必须匹配函数评价次数、初始化范围与维度。曲线横轴先用评价次数，再补墙钟时间；只比较“迭代了 30 代”会忽略不同种群和算法每代工作量不相同。[C11] [C12]

在 LLM 算法设计里，个体由数值向量变成程序，适应度由一次函数值变成一批问题实例上的求解表现。这个变化比“把随机变异改成提示词”更根本：评估开始涉及程序错误、运行时间、泛化和安全。

5.5 Attention 与代码生成：只保留真正要用的数学

单头注意力可以写成：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

令 $Q, K \in \mathbb{R}^{n \times d_k}$，$V \in \mathbb{R}^{n \times d_v}$，则注意力矩阵形状为 $n \times n$，输出为 $n \times d_v$。在自回归生成中，掩码 M 阻止当前 token 读取未来 token。多头、残差、归一化与前馈层需要理解，但第一轮不必实现分布式训练。[P18] [V03] [V10] [V11]

语言模型生成代码，实质上是在条件上下文下生成 token 序列。语法正确、格式遵从、算法可行和优化效果是逐级要求，不能从“next-token 预测”直接推导出程序正确性。temperature 改变采样分布，不是一个保证“创新程度”的物理旋钮；更高温度可能同时提高差异性和错误率。

区分四件事。提示与上下文学习改变输入，不必改变模型权重；监督微调用示例程序更新权重；RL 用奖励更新策略；外层进化筛选程序可以完全不更新模型权重。EoH 初始复现不需要先掌握大模型微调。自然语言 thought 是候选的外显说明，不是对模型内部真实推理机制的证明。[P02] [P08] [V07] [V08] [V09]

5.6 三个学习目标的数学区别

模仿学习。若示范数据为状态—动作对 (s, a^*)，离散动作策略可最小化 $\mathcal{L}_{\mathrm{IL}}(\theta) = -\mathbb{E} \log \pi_\theta(a^* \mid s)$。用于路由时，状态可包含未服务任务和剩余容量，示范来自一个明确版本的专家算法；数据生成成本需要计入。训练时见过的专家状态与自己执行后到达的状态可能不同，低训练损失不等于低路线代价。[V14]

强化学习。令 $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_t \gamma^t r_t]$，最简单的策略梯度估计使用 $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ 乘以回报或优势估计。若奖励只在整条路线完成后给出，信用分配比监督一个下一站选择更困难。第一轮可用负路线长度作为终局奖励，但可行性仍由环境/验证器处理；不能仅靠一个任意罚项保证满足容量约束。[V13] [V14]

进化程序。 EoH 式搜索保留高分程序、再通过模型提出变化，通常没有上述对生成模型参数的梯度更新。课程学习则安排被呈现的训练分布，本身既不等于模仿损失，也不等于策略梯度。只有明确了这三者的对象，才有意义讨论怎样组合。[P02] [P08]

亲手练习。用三种手写策略作为动作，在少量训练实例上构造策略表现表；先实现总选一个策略的基线，再做监督选择。之后把它改成逐步选择算子的序列任务，写明奖励和状态变化。前者接近算法选择/上下文决策，后者才包含真正的时序信用分配；不要用同一张效果图掩盖任务变化。

5.7 NAS 与程序合成：不删除，但不夺走主线

NAS 需要明确搜索空间、搜索策略、性能估计和最终重训。DARTS 通过连续松弛优化结构参数，是理解双层优化的好例子；它不是 “LLM 自动算法设计” 的同义词。教学实验可限制为少量 MLP 结构与固定训练预算，比较随机搜索和一个简单搜索策略。先验证公平评估，再决定是否进入昂贵的图像 NAS。[P14] [P15]

程序合成从规格出发寻找程序。测试用例可以提供证据，但有限测试通常不构成对所有输入的证明。反例引导的思想是：提出候选、寻找违反规格的输入、用反例约束后续搜索。把它迁移到优化启发式时，必须区分 “返回非法解” 的正确性反例与 “表现不佳” 的性能反例；后者不代表程序逻辑错误。[V18] [P17] [P24]

6 论文精读：把每篇读成一个可复现的实验

6.1 通用精读模板

每篇论文先写一页卡片：研究对象是什么；被 LLM 改写的最小单位是什么；模型看见哪些数据；评分器返回什么；搜索/训练/推理预算各多少；最强的对照是谁；哪项消融能支持核心机制；有哪些无法重建的细节。最后补一个反例：“在哪种场景下，这个方法不应比简单基线更好？”

顺序应是摘要与图示定位问题，方法确定变量与接口，实验核对比较对象，再回到引言检查主张是否越界。不要用 “这篇很强”“达到了 SOTA” 替代具体说明。每次阅读都留一个能运行或能核算的产物。

6.2 EoH：第一篇深度复现对象

核心理解。搜索对象是启发式，而不是一个问题实例的答案；候选携带自然语言说明和代码。探索与修改提示提出后代，真实评估决定保留。原论文包含在线装箱、TSP、流水车间调度；不要把后来仓库增加的 CVRP 等任务写进原论文实验范围。[P02]

必须重建的部分。用表格记录五种提示、父代抽样、初始种群、候选合法性处理、评估集、并列处理、停止条件与最大预算。原论文写每代每种策略产生一批候选时，不能用“代数相同”保证不同变体成本相同。你自己的现代模型复现需记录所有候选尝试和失败，而不仅是保留下来的程序。

最有信息量的对照。同模型的独立采样、无 thought 的代码进化、固定初始候选的搜索、以及完整 EoH。特别注意原论文中某些消融同时改变表示和提示集合；复现时需先忠实重建，再新增只改变一个因素的对照，不能把新实验反过来称为原论文设置。

验收产物。一页方法说明、一份与论文/当前仓库差异表、一个可恢复运行的实验目录、全部候选的原始记录、冻结候选在独立测试集上的质量与失败统计。换了模型只能称“方法复现/现代模型迁移”，不能声称精确复现历史数字。

6.3 FunSearch：评估闭环比语言流畅更重要

核心理解。语言模型负责提出程序，评估器提供可计算信号，程序库与搜索过程持续保留有用候选。它研究 cap set 和在线装箱，不是指南写的矩阵乘法系统。[P03] [C05]

阅读重点。找出人类固定了哪些程序骨架、输入规格和评价目标；检查候选如何进入程序库、怎样避免所有候选变得相似；说明“发现一个数学构造”和“得到对任意输入正确的算法”需要不同证据。

复现边界。第一轮在同一个自建装箱评估器上实现 FunSearch 式对照，只能叫机制对照；要声称论文复现，还需对齐原任务分布、程序库机制、模型与预算。官方公开代码不自动包含你运行所需的全部服务和安全隔离。学习时先跑作者给出的已生成函数，再决定是否进行大规模生成。

6.4 ReEvo：反思必须由行为证据约束

ReEvo 将反思文本纳入进化过程，提供一个研究“反馈如何帮助后续生成”的清晰入口。读论文时区分短期/长期反馈的来源与作用，不把 reflection 自动等同于数值梯度。[P04] [C06]

可执行问题是：同样的候选对比，给分数、给简短失败摘要、给结构化反例，哪一种在相同 token 与候选评估预算下更有效？若复杂反思只是花更多 token 采样出更好的程序，不能把收益全部归于反思机制。还应检查反馈文本是否忠实于实际失败，不让 LLM 自行编造原因。

6.5 LLM4AD：学习接口组织，不复制框架体量

它把方法、任务与模型接口组织起来，是阅读现成系统的合适入口，但平台论文不是一个新的单一搜索算法。先用它找到任务评估器与方法调用点，再决定哪些设计可以借鉴。[P05] [C03] [C04]

必须逐项核对 README、`pyproject` 或依赖文件、例子和安装版本。本次 README 要求 Python 不低于 3.9 且低于 3.13；这不影响本册教学代码在另一 Python 版本通过测试，但不能据此宣布 LLM4AD 在该环境已验证。平台写“安全评估”也不免除你针对不可信生成代码的安全审查。

6.6 AlphaEvolve 与 OpenEvolve：方法启发和原版复现分开

AlphaEvolve 强调通过评估反馈持续改进代码，覆盖科学与工程问题；论文是白皮书，作者报告的具体成果应与它的任务、评估器和预算一同阅读。学习价值在于理解多种评估信号如何驱动代码改进，而不是复制一个产品名称。[P06]

OpenEvolve 是独立社区实现，能帮助你理解程序演化工程；它不是 DeepMind 原版源码，也不能仅凭名称相似就作为原版结果已复现的证据。第一轮不同时部署它、EoH 和 LLM4AD 三个大系统，避免把接口调试当成方法研究。[C08]

6.7 必须纳入创新判断的近期研究

EoH-S 已把互补启发式集合本身作为设计对象；HeurAgenix 涉及启发式生成和选择；FunTune 把进化反馈用于 RL 微调生成器；ASRO 预印本研究求解器与实例生成器共同演化；ES-AHD 是本次查到的近期进化策略相关预印本。它们意味着 “加集合”“加选择器”“加 RL”“加课程学习” 都不能直接作为未有人做过的创新主张。[P07] [P08] [P09] [P10] [P11]

这里的核验深度并不相同：EoH 是方法正文与示例层面的重点审读；近期条目主要用于确认先例和收窄问题，尚未独立复现实验。把预印本的作者主张、会议正式发表和本地实测结果分列，不混成一张确定无疑的排行榜。

6.8 建议的最小必读集合与补充集合

主线先读 EoH、FunSearch、ReEvo、LLM4AD，再读综述整理地图。讨论创新前加读 EoH-S、FunTune 及与你的机制直接相关的近期工作。Transformer、CodeGen、AlphaCode 与 NAS 论文用于解释基础和边界；不需要把每篇历史模型论文都做全量实验。原指南的知识覆盖通过概念关卡完成，而不是靠把 20 篇论文都读到同一深度。

7 视频、课程与社区：按问题观看，而不是按播放量收藏

7.1 资源核验方式与访问边界

本次检索覆盖作者和实验室主页、课程官网、arXiv 与会议/期刊页面、GitHub 官方与社区仓库、Hugging Face 课程、YouTube 及 B 站的官方课程入口。它是有主题边界的资源筛选，不是 “全网穷尽式系统综述”。保留的条目需要有明确教学作用；资源表记录一手来源、核验深度和访问限制。个人转述、营销式 “最强教程” 和无作者来源的搬运合集不承担技术证据。

资源编号只用于引用和查找，不是开发阶段代号。V 类包括录像、课程、配套教材、讲义与媒体入口；**不是每个 V 编号都对应一门独立视频课**。同一报告的活动页与录像页分列，是为了提高可找回性，不重复计数。播放器可见、公开可下载、国内网络可访问和能否完成作业，是四个不同条件。本次没有逐帧观看全部视频，也没有实测你的校园网。

以下建议学时包含暂停推导和练习，不是视频官方时长；是可调整的学习预算。看不懂时先做最小例子，能够完成验收就跳过重复讲解，不用以 “刷完整个课程” 证明学习进度。

7.2 基础课程：直接服务本手册实验

学习问题	首选资源与观看范围	建议投入与验收
怎样独立运行和排错	Missing Semester [V02]: Shell、Git、调试、性能分析	4—6 小时；从干净目录重建环境，保存一次完整错误复现，不以截图代替日志
为什么贪心不保证最优	MIT 6.006 [V01]：图搜索、最短路、动态规划、复杂度	8—12 小时选学；解释 Dijkstra 与 TSP 求解器为何解决不同问题，完成微型 TSP 穷举
矩阵运算到底在做什么	3Blue1Brown [V15]建立直觉，动手学深度学习 [V03] [V04]做张量练习	4—6 小时；不用自动广播“碰巧跑通”，手写每个矩阵维度
随机实验如何比较	STAT 110 [V16]：随机变量、条件概率、期望与方差	6—8 小时选学；区分一次搜索中的候选数与独立搜索重复数
梯度从哪里来	Karpathy [V10]中的 micrograd，或中文 D2L 自动微分	6—10 小时二选一；手算小计算图，再与自动微分核对
约束和目标如何区分	Boyd 教材 [V17]：建模、约束、拉格朗日与对偶的入门部分	3—5 小时；解释不可行解不能仅因路径短就获胜；无需先读完整本凸优化

观看顺序。 Python 能完成本册测试的人，从图算法与随机实验开始；不会建虚拟环境的人先做工具课。线性代数、概率和代码实践交错学习，不把三门完整课程堆在 LLM 实验之前。

7.3 LLM 与代码生成：中文主线、英文补充

中文主线。用 D2L 的注意力章节解释公式与代码，再用李宏毅 2025 课程解释预训练、对齐、后训练和推理模型的差异。[V03] [V04] [V05] [V06] [V07] [V08] [V09] 第三讲偏内部机制剖析，不应代替从张量维度开始的注意力教学。第五至第七讲之后，写一张“模型参数有没有变化”的对照表：提示、检索、程序进化、监督微调、RL 微调分别改变什么。

实作入口。Hugging Face 课程用于分词、模型加载、生成参数和数据流程；Karpathy 课程用于看清小模型从数据到损失的过程。[V10] [V12] 第一次生成实验只需要理解输入 token、输出 token、采样和解析，不以训练一个大模型作为前置条件。代码模型历史论文用来认识能力和评估边界，不强行部署指南中的所有旧模型。

深入选修。 CS336 用于分词、训练细节和语言模型实验设计。[V11] 建议先用 6—10 小时选看相关课次；只有明确研究生成器训练的人，才追加大规模训练与系统优化作业。课程中使用的模型、工具版本和公开作业授权，应以所选学期页面为准。

验收。给同一个任务建立可复用提示，保存原始输出及解析结果；解释为何温度变化会改变输出分布，但不会自动改变模型权重；能指出“返回了完整代码”与“代码通过独立评估”的差别。

7.4 与自动算法设计最直接相关的四组材料

EoH 与 LLM4AD。先读 EoH 方法正文，再看 LLM4AD 首页的 TSP 演示及作者提供的教程材料。[P02] [V21] 演示用于观察输入、生成、评估和结果的关系；CEC/EMO 链接是教程幻灯片，不能因为名称叫 tutorial 就说有完整录像。第一轮用 2—3 小时串起论文图、README 与任务接口。

FunSearch 作者报告。Harvard CMSA 活动页能核对报告人、日期和题目，另有录像入口。[V19] [V20] 观看时只记四件事：什么被固定、什么被生成、怎样评分、结果怎样验证。随后拿在线装箱做一次对应分析，避免停留在“AI 发现数学”的故事层面。

算法配置教程。 AutoML 2023 的配置教程说明如何在给定参数空间内进行搜索；它正好能纠正“凡是让 LLM 调参数就是生成新算法”的混淆。[V22] [V23] 选学 3—4 小时，建立一个不使用 LLM 的随机参数搜索，作为后续公平对照。

语言模型与优化教程。 AutoML 2023 的 Language Modelling for Optimization 提供跨领域理解入口。[V24] 适合在完成第一次装箱闭环后观看；带着自己的输入/输出/预算表去对照，而不是第一次接触算法时直接进入宏观演讲。EoH-S 官方页面还提供 Video/Slides 入口，用于后期讨论集合设计先例。[V25]

7.5 强化学习、模仿学习、课程学习何时进入

David Silver 课程优先看 MDP、价值函数和策略梯度；CS285 优先看模仿学习及策略优化的相应课次。[V13] [V14] 主线为理解预留 8—12 小时，不要求把深度强化学习全课修完。

最先要回答的是：**谁在学习，动作是什么，奖励什么时候给，训练后哪里发生变化？** 学习车辆下一站、学习选择局部搜索算子、学习生成程序，是三个不同问题。模仿学习可先模仿强启发式的选择；课程学习可改变实例难度或训练采样，但不能把测试集表现直接作为调课表的反馈。只有一个接口、一个训练对象和一个明确基线稳定后，才进入训练实验。

若研究只是生成一个离线启发式函数，RL 并非必需。EoH 原方法中的外部进化也不是 RL 微调；FunTune 则提供了研究参数更新的相关先例。[P02] [P08] 不为“融合更多方法”而增加无法单独验证的训练环节。

7.6 社区使用方法：从提问到证据

对 EoH/LLM4AD，优先查作者 README、历史版本及 Issues；对神经组合优化，查 RL4CO 的文档和仓库讨论；对程序进化，比较 FunSearch 原始代码与 OpenEvolve 社区实现。[C02] [C03] [C05] [C08] [C17] 社区答复是排错线索，不能替代论文实验或许可证文本。

一份有效提问包含：仓库与 commit、Python 及依赖、最小命令、完整异常、期望行为、已经排除的原因。不要把 API 密钥、付费账户信息、私有数据或完整机密日志贴进公开 Issue。发现答案后在自己的实验记录中写下对应版本和变更，避免下次升级又遇到同一个问题。

每周最多预留 1 小时集中跟踪社区。新增资源必须说明它会改变哪一个实验决定；没有具体作用的收藏不进入主学习清单。资源失效时先回到作者机构/会议的上一级入口，再查归档，不使用来历不明的安装包替代。

8 源码阅读与资源选择：先找到评估器，再找生成器

8.1 仓库的职责与使用顺序

资源	在本计划中的职责	第一遍看什么；暂时不做什么
EoH [C02]	主方法复现入口	README 版本提示、装箱/TSP 例子、评估方法；不同时适配几十种任务

资源	在本计划中的职责	第一遍看什么；暂时不做什么
LLM4AD [C03] [C04]	对照接口与统一任务组织	方法、任务、LLM 接口；不先开发 GUI 和复杂服务
FunSearch [C05]	程序库与评估反馈的机制对照	程序骨架、评估和候选保留；不把社区复写视为原版结果
ReEvo [C06]	反思反馈对照	反馈来源、生成提示与演化过程；不只比较最终最好分数
EoH-S [C07]	集合研究的必要先例	集合评价、互补性与测试流程；不直接宣称“集合”创新
OpenEvolve / LLaMEA [C08] [C09]	扩展工程与优化器设计	选一个进行接口比较；不与主框架同时大改
HeurAgenix [C10]	生成和选择的组合先例	生成器、选择器及其训练/测试边界
DEAP [C11] [C12]	经典进化实现参照	fitness 方向、复制、变异后的失效与重评估
PyVRP / HGS-CVRP [C13] [C14]	后续路由强基线	输入格式、成本精度、时间预算、版本；不是直接替代自己的问题定义
ALNS [C16]	邻域搜索机制练习	destroy/repair、接受准则及算子选择；不预设为最终研究唯一骨架
RL4CO [C17]	神经组合优化拓展	环境、策略、训练器、数据分布与评估；先跑小例子再考虑训练
HumanEval / OR-Tools [C18] [C15]	正确性评估边界与路由参照	不可信代码安全警告、求解状态；通过代码测试不等于优化效果好

当前 PyVRP README 以迭代局部搜索描述求解器，不能把早期论文的 HGS 描述不加版本说明地套到当前代码。HGS-CVRP 则是另一独立基线。选择谁取决于问题约束和预算，不取决于仓库名气。[C13] [C14]

8.2 原指南其余代码资源的去留

BigCodeBench 与 APPS 用于代码任务和功能正确性评估, 不替代优化启发式 benchmark; 本次 BigCodeBench 页面已显示归档状态, 应固定版本并评估依赖维护风险。[C19] [C20] CodeGen、StarCoder 和 CodeLlama 保留为历史模型与推理代码阅读入口，按模型卡和许可证决定是否使用，不强制全部部署。[C21] [C22] [C23]

NASBench 的可核对入口为`google-research/nasbench`，它用于 NAS 基准学习；指南的泛化仓库名不能直接作为安装依据。[C24] 与 AlphaCode 相关的公开 CodeContests 仓库提供数据和相关材料，不等于完整 AlphaCode 训练系统的可复现发行版。[C25] 这一组作为分支练习，主线仍是启发式生成、评估和搜索。

8.3 真实可定位的入手点

EoH。当前 README 说明：新 API 是 `LLMConfig / EoH / BaseProblem`，旧 `Paras / eoh.EVOL` 对应旧版本。已核对的例子目录包括`examples/bp_online`、`examples/tsp_construct`，当前也有多函数、类形式与 CVRP 相关例子；这不等于原 ICML 论文已经研究了所有新增任务。[C02]

LLM4AD。已核对的 README 指向`example/online_bin_packing/online_bin_packing_tutorial.ipynb`、`example/tsp_construct/run_eoh.py` 和`example/vrptw_construct/run_eoh.py`。其公开示例使用 `OBPEvaluation`、`HttpsApi`、`EoH` 及 `EoHProfiler`。[C03] 本册不编造指南中不存在的 `main.py --problem obp` 启动方式，也没有声称这些上游脚本在本环境运行通过。

通用源码阅读顺序。从一个输入实例开始，追踪到评价结果：输入检查 → 人工基线 → 生成函数调用 → 独立验证 → 聚合指标 → 选择/保留 → 日志 → 下一次提示。只在必要时进入网络客户端、并行调度和可视化。每读完一段，用一句话说清它是否会影响科学结论；没有这个判断，逐行阅读也可能只是浏览。

8.4 版本、环境与复现记录

Ubuntu 22.04 作为目标系统时，为教学代码、EoH/LLM4AD 和神经模型训练分别建立虚拟环境。前两者先使用项目支持的 Python 版本；不要因系统 Python 或某个 GPU 包的要求，把全部仓库强行放进一个环境。指南并不要求一开始训练 LLM。[C02] [C03]

每次克隆后先记录版本，再安装。下面是通用记录命令，**不是对所有仓库的统一安装脚本**：

```
git rev-parse HEAD
git status --short
python --version
python -m pip freeze > requirements-observed.txt
```

复现实验应固定实际 commit、数据校验值和配置，而不是长期跟踪 `main`。README 文件的 blob SHA 只标识该文件内容，不能当成整个仓库的 commit。锁文件、容器镜像和数据 manifest 各自解决不同问题，不能相互替代。升级后先跑相同的微型实例，再解释数值差异。

8.5 算力与费用：先用可测量的预算说话

教学代码和经典小实例以 CPU 为主。调用远程 LLM 时，程序评估仍可在独立 CPU 环境执行；本地模型部署才需要进一步考虑显存和推理吞吐。8B 参数以每参数 2 字节存权重，权重本身约 16GB；这只是数量级计算，不包含 KV 缓存、激活、运行库及量化元数据，不能据此保证某张卡可运行完整任务。

不在本册固定易变的 API 价格、模型排行榜或并发额度。首次联网生成只开一个并发 worker，设置硬预算上限；失败调用、重试、解析失败和无效候选都计入费用。先测 20 次调用的 token、耗时及失败率，再决定扩大预算。这里的 20 是试运行的管理预算，不是统计充分性门槛。

手机适合阅读、标注、写问题清单及查看远程结果；完整编译、测试和安全执行放在已配置的电脑或云端工作目录。作者提供的 Colab 可以用于体验，但本次没有替你运行其笔记本，也不保证免费额度、网络连通性或第三方平台安全边界。不要为了得到一张演示图把密钥写进公开 notebook。

9 端到端最小系统：把可以信任的部分先建好

9.1 系统的数据流与责任边界

环节	输入 → 输出	不应越过的边界
问题与实例	任务规格、数据 → 验证后的实例	坐标、单位、约束和不可达关系必须明确
人工算法与精确参照	实例 → 解及求解状态	微型精确结果与大规模最好已知结果分开
任务包装器	实例、候选函数 → 完整解	可行性、终止和回退由可信代码负责
候选生成	任务接口、训练反馈 → 候选源码	不向生成器暴露测试解、评分器写权限或密钥
隔离执行	源码、受控输入 → 运行结果	独立受限环境；超时、崩溃和安全拒绝均留记录
独立评估	完整解 → 正确性、可行性、质量、成本	不相信候选自报的目标值
搜索与保留	训练评估 → 下一代/候选库	验证集只按事先规则用于少量选择
测试与报告	冻结方法 → 测试结果与图表	查看测试结果后不得偷偷继续选择同一结论

第一版只开放一个纯评分函数，例如 `score(item, residuals)` 或 `priority(features)`。它返回分数，不自行决定文件访问、实验配置和结果写入。任务包装器用合法动作掩码、确定性的并列规则及回退策略生成完整解。接口简单不等于科研简化；它能使方法差异和失效来源更容易被识别。

9.2 候选对象、运行记录与缓存

一个候选至少记录：源码 hash、父代 hash、生成方法、提示 hash、模型标识、调用参数、原始返回文本、解析状态及创建次序。一次评价至少记录：候选 hash、实例 hash、求解器版本、随机种子、预算、运行状态、独立目标值、可行性、时间和资源消耗。不要只保存最后一个 “最佳.py”。

状态应能区分成功、不可行、超时、内存不足、语法错误、接口错误、崩溃、安全拒绝及 API 失败。API 失败可能发生在候选产生之前，不能捏造成某个候选的差质量。聚合时同时报告过程可靠性与成功后的解质量。

缓存键至少覆盖代码、数据、配置、种子和评估器版本。完全相同的确定性评价可以缓存，但节省了多少计算要如实说明；随机评价不得因忽略 seed 被错误复用。代码文本重复与行为相同是不同层次的重复，后者需要预先规定的探针实例或行为特征。

9.3 搜索闭环的最小伪代码

```
读取冻结的问题规格、训练集和总预算
用人工基线建立可运行的初始候选
当尚有预算时：
    从候选库选择父代或上下文
    生成一个候选；记录调用、重试与原始输出
    检查接口；在隔离环境运行微型安全/可行性测试
    若未通过，记录原因，不丢弃失败计数
    若通过，在训练实例上评价并计入预算
    按事先规定的保留规则更新候选库
按预定规则使用验证集选一个候选或一个固定集合
冻结源码、选择规则与配置
一次性运行保留测试集；导出全部状态与独立指标
```

这段流程不是新的算法贡献，而是任何方法比较都需要的实验骨架。最初可用固定的手写候选代替网络生成器，验证整条数据流；这样的 dry-run 必须标为测试模式，不能计为 LLM 实验成果。

9.4 生成代码的安全执行不是一个 timeout 参数

AST 检查、限制导入和子进程超时可以减少部分错误，却不是可靠的恶意代码安全边界。HumanEval 官方仓库也明确警告，不可信代码不能未经隔离直接执行。[C18] 学习中把候选视为不可信输入：主程序持有 API 密钥，评估 worker 不继承密钥；网络关闭；输入只读；可写临时目录有限；限制 CPU、内存、进程数与运行时间；禁用特权和宿主机敏感挂载。

采用容器也不自动等于强安全；需要按风险选择额外隔离或虚拟机，并在运行未知代码前完成配置审查。此交付包**不实现生产级沙箱**，也没有包含运行任意 LLM 源码的 `exec` 入口。教学代码只运行随包附带、可人工审读的函数，避免把未验证的安全组件包装成科研能力。

9.5 随包教学实验：已经运行了什么

`teaching/lab.py` 与 `test_lab.py` 提供最小纠错实验，使用本地 NumPy 与 pytest，不需要 API、外部数据或 GPU。运行方法：

```
cd teaching
python -m pytest -q
python lab.py
```

检查项	本次实测	能支持什么；不能支持什么
单元测试	29 项通过	教学函数通过这些测试；不是全面正确性证明
原指南关键错误的最小重现	`TypeError`	新一代未评分便比较会出错；不是运行了原 Word 中的完整工程
修正后的 GA	744 次目标评估；历史最佳不升	评分顺序、预算计数与精英保留有效；不是某种优化器达到 SOTA

检查项	本次实测	能支持什么；不能支持什么
正方形 TSP	最优 4.0；交叉回路 4.828427...	微型精确参照与独立成本核验正确
装箱例子	best-fit 为 2 箱，next-fit 为 3 箱	策略和实例影响质量；不能推导总体优劣

本次教学环境为 Python 3.13.5、NumPy 2.3.5、pytest 9.0.2；具体平台信息和日志随包保存。上游 LLM4AD 的版本要求不同，因此这些测试不能冒充其安装验证。GA 数值只用于回归检查，不是跨平台位级一致性的承诺。

9.6 从教学实验到论文复现的四次升级

先把教学装箱评估改成可载入固定实例集；再接入一个人工评分函数和一个固定候选库；随后接入一种真实 LLM 生成方法，记录所有调用；最后按论文设置建立基线、独立重复和测试分割。每次升级只增加一个影响结果的因素。任一步出错都能回退到已有可信结果，而不需要重建一个庞大平台。

10 研究级实验协议：让改进经得住重新运行

10.1 数据划分与泄漏控制

至少区分设计训练集、方法选择验证集和最终测试集。训练集可用于反复进化；验证集只服务预先规定的超参数或候选选择；测试集在方法冻结后使用。若看了测试集又修算法，应建立新的独立测试证据，或如实把该集合降为开发数据。公开 benchmark 可能已经进入模型训练，不能声称仅靠随机切分就消除了预训练污染。

随机切分不是唯一方案。由同一地块、同一母实例或同一生成参数派生的样本应按组划分；防止旋转、尺度变化、点位扰动后的近重复实例跨集合。合成数据记录生成器、分布、seed 与过滤条件，现实数据记录来源、许可、单位与预处理。删除失败或困难实例必须留理由，不能以“清洗”为名提高分数。

分布外测试至少分别考虑规模、几何/需求分布和约束变化，不把几种变化混成一个难以解释的 OOD 集合。规模外推不等于跨任务泛化；TSP 到 CVRP、CVRP 到 VRPTW 也不是同一层次的问题。[P02] [P07] [C17]

10.2 必需基线和逐步增加的比较

要判断的主张	最少需要的对照	必须控制的因素
LLM 生成有效	人工规则、同预算独立 LLM 采样	模型、提示信息量、调用/评估成本
进化优于重复采样	独立采样、固定候选库、完整进化	相同候选数与总时间；均报告失败
自然语言反馈有效	仅分数、随机/打乱反馈、真实反馈	反馈 token 与生成预算，不能只控制迭代数
设计比配置更有效	人工结构的随机/标准参数搜索	可调空间、试验数与开发成本
集合/选择器有益	最佳单一算法、静态集合、随机选择	选择器额外成本及训练信息
学习方法有益	手写策略、无训练对照、简单学习器	数据、训练和推理总成本
农业约束有意义	同任务无该约束/不同强度的敏感性实验	不更换任务定义后直接比较总路长

不是要求一次跑完所有基线，而是每提出一条主张补上对应证据。小问题先建立精确参照；大问题采用强公开求解器及明确的最好已知结果，不把未证明的参考值叫最优值。[C13] [C14] [C15]

10.3 预算：同时回答开发效率与使用效率

报告三条成本轴：模型生成的 token 与费用、候选评估次数与墙钟时间、冻结算法在新实例上的推理时间。相同迭代数不代表公平；失败重试和无效程序也是方法成本。并行硬件、线程数和终止策略一致或清晰标注。

若方法在部署前花费搜索成本 C_{search}，在 M 个实例上执行，每个实例开销为 C_{run}，则摊销成本是

$$\bar{C}(M) = \frac{C_{\text{search}}}{M} + C_{\text{run}}.$$

这不是给昂贵设计辩护，而是把“生成很贵、使用很快”的取舍写清。与参考法比较时，应代入两者各自的搜索/训练与使用成本，给出不同 M 下结论是否改变。一次性任务和重复作业任务可能得到不同推荐。

第一轮可规定 20 个候选用于接口试运行、100 个候选用于方法排错；这些是管理上限而不是论文充分预算。正式实验预算根据初步方差、成本曲线和研究问题确定，不能先用测试成绩决定某个方法多跑多少轮。

10.4 指标的定义与失败处理

正成本且参考值大于零时，可报告相对差距

$$g_i = \frac{f_i - f_i^{\text{ref}}}{f_i^{\text{ref}}} \times 100\%.$$

必须说明参考是精确最优、下界还是最好已知解；对零/负参考值使用预先定义的绝对差或其他归一化，不靠任意极小数掩盖问题。装箱等离散目标同时报告原始箱数；路径任务同时报告单位及成本定义。

可行性优先单独报告。只对成功实例计算均值会产生幸存者偏差；至少同时给出成功率、超时/不可行数量、共同成功实例的成对比较和对全部实例的明确处理。若使用惩罚值，必须预先说明依据，并给出敏感性分析，不能让某个随手选的罚常数决定结论。

候选级正确率与最终算法质量分开。候选失败率低可能来自保守搜索，不必然产生更好的解；最终最优分数高也可能来自极高试错成本。报告任一单指标都会漏掉方法的重要性质。

10.5 随机性、独立单位与统计推断

至少区分生成搜索 seed、下层求解器 seed、实例 seed 和训练 seed。一个搜索过程产生 1000 个候选，仍不是 1000 次独立搜索。方法在同一测试实例上配对比较；多个运行共享训练数据的依赖也要说明。[P22] [P23]

先进行小规模试验估计变异，再确定正式重复数。若预算只允许 3—5 次独立搜索，应报告分布和不确定性，不用“取最好一次”代替稳定性。作为可执行起点，本计划建议先做 5 次独立搜索的探

索性比较；是否增至 10 次或更多由区间宽度与计算预算决定，不能预先宣称某个次数具有统计充分性。

报告效应量、成对差值和置信区间；bootstrap 应按真正独立单位重采样，例如先按搜索运行、再按实例组分层，而不是把所有 run-instance 行当成独立样本。多数据集排名检验有适用条件，不能机械套用 Demšar 论文；多重比较和重复查看结果应提前考虑。实用改进门槛应来自应用容忍度或成本，而不是为了显著而事后指定。[P23]

10.6 消融、停止与诚实的负结果

第一篇报告只消融最关键的两个因素，例如反馈类型与多样性机制；其余保持固定。既跑 “移除部件”，也验证简单替代方案能否达到相同效果。若同时换模型、提示、训练数据和搜索预算，结果不能归因于其中一个因素。

在正式运行前写下停止条件：预算耗尽、评估器错误、安全问题或预先约定的统计规则。安全/正确性问题触发立即停止；性能不够好不会触发偷偷换测试集。对失败方法写明适用范围、失效实例与成本，负结果也可以成为可靠的复现报告。

10.7 应交付的图与表

推荐六类结果产物：质量—总成本曲线、各类失败比例、训练/验证/测试差距、规模及约束外推、关键机制消融、具体成功与失败案例。曲线展示多个独立运行的区间，不只画最好轨迹；对数轴、截断纵轴和归一化方式要解释。多目标任务可绘 Pareto 前沿，但只有目标存在实际取舍且单位/可行性明确时才采用，不能因图好看就增加目标。

图必须能从原始记录一键重建。自动生成的截图或任意模拟曲线只能标为示意，不进入结果区。本册没有伪造论文实验图；随包数值仅来自明确列出的教学运行。

11 24 周主线与前 14 天安排

11.1 时间的使用原则

主线按每周 14 小时安排，共 336 小时，是建议预算，不是 “24 周后一定能发表” 的保证。每周用约 4 小时阅读/视频、7 小时代码/实验、2 小时结果检查、1 小时写作与交流；临近复现实验可把观看时间转给运行分析。基础缺口允许前置 0—8 周；主线后另设 8—12 周研究扩展，不把扩展工作藏进同一份 336 小时计划。

不固定某天必须连续学习多久。每周至少保留两次不看参考实现的独立推导或编程，并保留一次从失败中解释原因的记录。章节、录像和代码共同围绕一个可验收问题，不要求每天同时学五门课。

11.2 每周产物与过关条件

周次	学习与实验任务	当周必须留下的证据
第 1 周	阅读纠错章；运行教学包；补 Shell/Git [V02]	环境记录、29 项测试日志；口述原指南三项事实错误与一项代码错误
第 2 周	贪心、图搜索与微型 TSP [V01]	独立实现/解释穷举参照；区分路径可达性与访问顺序

周次	学习与实验任务	当周必须留下的证据
第 3 周	GA 的选择、变异、重评估 [C11] [C12]	单目标方向明确；预算计数；加入 dim=1 和零代数测试
第 4 周	随机性、复杂度与公平比较 [V16]	对同一实例成对比较；说明为何候选数不是独立重复数
第 5 周	注意力、token 与自动微分 [V03] [V04] [V10]	手写形状推导；小计算图梯度核验；不依赖大型 GPU
第 6 周	预训练、对齐、提示与采样 [V05] [V06] [V07] [V08] [V09] [V12]	一张参数是否更新表；一次有预算上限的生成记录，或明确的离线解析练习
第 7 周	程序合成、规格、测试与 NAS [V18] [P14] [P15]	小表达式合成练习；NAS 与启发式设计异同比较；安全边界说明
第 8 周	复盘三种教学任务，建立接口合同	装箱/TSP/CVRP 微型问题规格；手写基线与独立验证，不急着训练
第 9 周	精读 EoH 正文与当前 README [P02] [C02]	五个搜索提示算子的准确说明；原版/当前版差异表
第 10 周	追踪一个装箱例子的完整调用链	一个实例从输入到 fitness 的追踪笔记；源码定位与版本记录
第 11 周	接入受限生成与可靠性记录	小预算端到端运行；所有失败、重试和 token 保留；评估环境安全审查
第 12 周	做 EoH 式首次方法复现	可恢复实验目录；与人工及独立采样对照；明确尚未对齐的论文设置
第 13 周	学习 FunSearch 程序库机制 [P03] [C05] [V19]	在同一包装器下的机制对照；与原版复现边界说明
第 14 周	学习 ReEvo 反馈机制 [P04] [C06]	分数/结构化反馈接口；打乱反馈对照设计
第 15 周	读取 LLM4AD 接口 [P05] [C03]	现有实现与平台的适配差异；不重新搭建一个完整 GUI
第 16 周	冻结第一次比较协议，独立重复	数据分组划分、预算与运行 seed 清单；质量/成本/失败率表
第 17 周	从 TSP 转向 CVRP 约束 [C13] [C14] [C15]	小规模容量与回库核验；强基线输入对齐；不可达/不可行案例
第 18 周	区分算法选择、配置、设计 [V22] [V24]	同一人工结构的配置基线；说明自动生成到底新增什么自由度
第 19 周	RL 与模仿学习入门 [V13] [V14] [C17]	为一个小问题定义状态/动作/奖励；简单模仿或表格学习练习
第 20 周	课程与分布外泛化；查近期先例 [P07] [P08] [P09] [P10] [P11]	一个不使用测试集反馈的难度采样方案；创新重合检查表
第 21 周	定义农业路径与任务问题，不先固定场景	单位、约束、地图、任务点与评估指标；导师可阅读的两页问题说明
第 22 周	选择一条研究假设并做可行性试验	对照、预期证伪条件、算力预算；必要时放弃该假设而保留复现成果
第 23 周	整理复现报告与可重建图表	方法、差异、结果、不确定性和失败案例；全部图表可追溯
第 24 周	从干净环境复跑、口头答辩与修订	可运行打包产物；10 分钟讲解；不能回答的问题进入明确补学清单

第 12、16、24 周是建议的验收节点，不是冻结研究的行政门槛。若评估器不可信，第 13 周不进入新方法，而是继续修复；若基础已具备，可压缩课程周，把同等精力投入更多独立复现和失败分析。

11.3 前 14 天：把“开始学习”变成具体动作

天	当天重点（约 2 小时）	完成标志
1	阅读本册 1—3 章，对照原指南做更正	至少写清四角色、FunSearch 和文献仓库三处问题
2	运行教学测试并阅读日志	说明每类测试在保护什么，不能只说 “29 passed”
3	独立重现空适应度错误	写出先评分后选择的正确生命周期
4	手算装箱序列，再改变到达次序	区分离线排序、在线决策和重新装箱
5	手算正方形 TSP 并检查交叉回路	解释最优参照和独立计分器的用途
6	看 MIT 图算法相关片段并画小图	区分最短路与 TSP，不用最短路长度代替全任务成本
7	不看源码讲解 GA 与装箱；写半页总结	讲出一个失败例子和一个尚不能解释的问题
8	手写 TSP 置换验证器	重复节点、漏节点、越界输入均被发现
9	给 TSP 做重标号和尺度测试	成本按预期变化，解的标签变化不造成伪改进
10	画一个容量 10、需求 6/6/4 的 CVRP	能判断哪些合并可行，理解回库不是错误
11	阅读 EoH 摘要、方法图及任务说明	用自己的话说清 “程序是个体，路线是解”
12	看 README 并记录版本信息	识别新版 API 与旧教程；不盲装所有依赖
13	写生成函数的输入输出合同	不允许候选修改评分器、数据划分或实验日志
14	进行 10 分钟自测并修正前两周记录	能回答概念、代码、实验、公平性各一个问题

第 1—2 周总量仍按 28 小时计；不要在这张日程之外再叠加一套 14 小时周任务。未接入真实模型前，离线解析和手写候选只能记作接口练习，不能写成完成 LLM 算法设计。

11.4 基础补学与深入扩展

需要前置补学时。用一组能力题诊断，而不是刷题数量：能否独立编写纯函数与测试；能否说明时间复杂度；能否解释期望和方差；能否读懂矩阵形状；能否从终端重跑一个程序。每个缺口补 1—2 周，最多按需要安排 0—8 周；已经会的项目直接免修。

研究扩展 8—12 周。前 2 周补齐最相关先例及差异；随后 2—3 周实现唯一核心机制；再用 2—4 周做固定协议的重复与外推；最后 2—3 周复核、作图与写作。几个区间不是要求取所有上限，按总预算协调。研究没有出现可靠提升时，交付严谨复现、负结果和边界分析，不虚构创新。

12 研究候选与农业规划迁移

12.1 不把技术名词拼接当成研究问题

“LLM+RL+ 模仿学习 + 课程学习 +VRP” 只是技术组合，没有说明什么未知问题被解决。当前已有集合演化、启发式选择、生成器 RL 微调和求解器/实例共同演化的研究。[P07] [P08] [P09] [P10] 合理起点是识别一个现有方法没有充分验证的机制，再用对照验证；不是先给系统起名、再寻找证明它有用的数据。

下面三个候选都是**待文献深入核查、待实验检验的设计假设**，不是声称已发现无人研究的方向。主线结束时只选一个，不同时进行三套研究。

12.2 候选一：结构化反例反馈是否提高有效搜索效率

问题。与只有分数或自由文本反思相比，来源可核对的失败反例，能否在相同总预算下提高可行候选比例、降低找到有用启发式的成本？ReEvo 和程序合成工作已经研究反馈与反例相关思想，因此创新不能只写 “加入反思”。[P04] [P24]

最小实现。可信验证器把失败转成固定字段：违反的接口/约束、最小失败输入、候选实际输出、独立期望条件。失败缩减的计算也计入预算；生成器只看到训练实例中得到的反例。不可约束的自由文本建议留作对照。

核心比较。同一模型、同一候选接口，比较只给分数、给长度匹配的通用说明、给随机/打乱反例、给真实结构化反例。分别报告正确性、可行性、质量与总成本；控制失败数量差异带来的反馈 token 变化。

证伪条件。收益在 token 匹配后消失；只改善语法而不改善解质量；反例计算成本抵消搜索节省；换一种实例分布后没有收益。出现任一情况都需要缩小主张，而不是继续调图。

12.3 候选二：带回退机制的启发式选择能否减少坏情况

问题。不是再证明一个集合优于单算法，而是研究低成本选择器何时应该放弃自信预测、回退到稳健基线，以减少分布变化时的严重退化。EoH-S 与 HeurAgenix 是必须对照的先例。[P07] [P09]

最小实现。先固定一个小候选集合，提取求解前即可计算的实例特征，用简单树或线性模型预测相对表现；低置信度或超出训练范围时回退。特征开销、训练数据与回退阈值均由训练/验证确定，测试集不参加阈值选择。

关键边界。未经形式化证明的经验回退不能叫 “性能保证”；校准也不自动在任意 OOD 分布成立。至少对比最佳单算法、随机选择、始终回退、无回退选择器及不可部署的 oracle 上界。看最坏分位、尾部退化和平均成本，而不是只看平均路长。

证伪条件。选择器费用高于改进；简单单算法已经足够；回退几乎从不触发或几乎一直触发；收益只来自在验证集反复搜索阈值。此时合理结论可能是该算法池无需复杂选择器。

12.4 候选三：受控课程是否改善约束泛化

问题。在生成预算固定时，按规模、容量紧张度、障碍结构或时间窗紧张度安排训练实例，是否比固定混合采样更能产生跨分布有效的启发式？课程或共演化已有相关先例，必须与 ASRO 等区分；也不能把“从小到大”本身作为贡献。[P10]

最小实现。起初不训练实例生成器，只固定一个可解释的难度坐标和若干训练分布；比较均匀混合、固定递进和预先定义的自适应采样。难度应与实际可行性和求解成本相关，不能仅凭节点数命名“困难”。

学习方法的合理位置。模仿学习可初始化算子选择器，RL 可学习后续决策；但只有选定这一机制时才加入。先与不学习的课程比较，再研究学习课程是否值得；不同时更换启发式表示与生成模型。

证伪条件。所谓泛化只发生在相似的生成器上；收益来自更多见过的实例而非课程；困难分布造成大面积无效候选；测试集参与调整课程。若这些问题无法解决，就保留均匀混合基线。

12.5 农业任务如何进入，而不把主线带偏

农业迁移先写问题定义：机器人在哪种二维可通行空间中移动；任务是访问点、沿边服务、覆盖区域还是混合；任务需要多少作业资源；在哪里补给；存在什么时间或作业顺序约束；优化哪些具有实际意义的成本。只有这些条件成立，才能判断适合 TSP、CVRP、VRPTW、弧路由、覆盖规划或其他模型，不能先选名词再解释场景。

两个相连但不同的规划层。几何/图上的点到点规划提供可行转移与成本；任务规划决定访问哪些任务以及先后次序。若位置、朝向或入射方向影响下一段成本，单个静态距离矩阵可能不足，需要显式状态扩展或更合适的接口。矩阵元素对称、三角不等式成立以及所有点相互可达，都必须验证而不是默认。

从仿真走向可信实例。第一批场景用可控二维地图，包含通道、禁行区、任务点和补给点，明确长度、时间和容量单位。再引入来源与许可可核对的数据，按原始场景分组划分训练/测试。若没有真实作业验证，只能声称二维模型下的方法效果，不能把地图演示称为实际农业部署。

自动算法设计的位置。它可以改进候选动作评分、邻域排序、局部搜索调度或算法选择，但不能修改任务真值和评价规则来制造提升。几何层的错误不能通过更复杂 LLM 来掩盖。科研图首先展示地图与约束如何对应模型，再展示预算、公平对照及失败案例，不先追求“数字孪生”外观。

13 从学习产物到可审查的论文

13.1 论文结构与证据对应

写作部分	要回答的问题	最低可追溯证据
引言与相关工作	未解决的是哪一个具体问题	与最近先例逐机制比较；不能用“农业暂无应用”替代方法空白
问题定义	输入、约束、输出、目标和假设是什么	单位、边界条件、可行性验证器和微型实例
方法	新增了哪一步，为什么可能有效	伪代码、接口、预算计算；不把平台功能当成算法贡献
实验设置	哪些条件固定，哪些允许调参	数据分割、模型、版本、种子、超参数和成本账单

写作部分	要回答的问题	最低可追溯证据
主要结果	效果大小、不确定性和成本是什么	全部实例结果、独立重复、强基线与失败统计
消融与机制	收益是否真由该机制产生	单因素对照、简单替代方案、外推与案例
局限和复现	结论在哪些情况下不成立	负结果、偏差来源、权限/安全边界、可运行材料

不能先写“显著优于现有方法”再找实验支撑。开始写作时把主张分成已证明、实验支持、合理推测和未来工作四类；不同类型使用不同语气。顶刊要求是严谨性的目标，不是排版完成后的身份认证。[P22]

13.2 投稿前必须能够回答的八个问题

你是否能在不看代码时定义问题？评估器能否独立重算结果？原论文设置与改动是否透明？所有比较是否包含失败和完整成本？数据是否按真正的独立单位划分？新机制是否胜过简单替代？换分布后结论还剩什么？第三方是否能够用固定版本重建核心表格？

任何一项不能回答，都对应具体修复任务。工具数量、单元测试总数、代码行数、漂亮图和“多智能体”名称都不能代替这些答案。测试很多但验证错了目标，依然是不可信实验。

13.3 每周汇报与答辩表达

每周汇报只需一页：本周问题、做过的实验、实际结果、发现的限制、下周唯一优先事项。明确“读过”“运行过”“复现过”“自己实现过”的区别。没有运行的数据就不填结果格；文献中的数字必须注明来源，不能混入个人实验表。

十分钟讲解按问题定义、方法机制、评估可信度、结果与局限组织。讲到老师指南中的错误时，可以说：“我沿用自动算法设计的主线，但核对原论文和仓库后修正了引文、角色分类与运行接口。”这是对材料的学术核查，不是对提供材料的人作价值判断。

14 附录：概念自测与参考答案

14.1 20 个必须能独立回答的问题

问题	合格答案的核心，不是背诵模板
1. 一个 TSP 排列与一个启发式函数有何区别?	前者是某个实例的解；后者是可作用于多个实例的算法组成部分
2. EoH 中的个体是什么?	思想描述与可执行代码构成的启发式候选，不是必然为一条路线
3. 综述四角色是什么?	optimizers、predictors、extractors、designers；不是指南的 Programmer/Evolver 四分法
4. FunSearch 与 AlphaTensor 有何区别?	前者是 LLM 程序搜索及外部评估；后者研究 RL 发现矩阵乘法算法，任务与方法不同

问题	合格答案的核心，不是背诵模板
5. EoH 需要先微调一个 LLM 吗?	原方法不以微调为必需前提；外部进化不等于参数训练
6. 为什么 1000 个候选不是 1000 次独立实验?	它们共享父代、反馈和搜索历史，存在依赖
7. 为什么不能直接相信候选返回的 cost?	候选可能算错、遗漏约束或操纵评分；可信验证器必须重算
8. 最大化 fitness 能否优化最短路?	可以转换符号，但方向必须在所有选择/排序/报告中一致
9. 训练最好为何不等于测试最好?	搜索反复适应训练样本，可能过拟合；选择还会引入偏差
10. 相同代数为何不公平?	每代候选、重试、token、评估和并行资源可能不同
11. first-fit、best-fit 与 next-fit 有何区别?	选首个可行箱、最紧可行箱、只尝试当前箱；到达顺序会影响结果
12. 微型 TSP 穷举有什么价值?	提供可核验的精确参照，发现成本/可行性错误；不解决大规模复杂性
13. 距离矩阵何时可能不对称?	单行约束、方向状态或不同转移代价均可能导致方向相关；需建模核验
14. 为什么 HumanEval 高分不足以证明启发式好?	编程正确性指标与优化目标、泛化、成本不是同一评价
15. 一张 GPU 为何不代表能训练所有代码模型?	参数、优化器、梯度、激活、缓存和运行框架都有内存与计算成本
16. 什么情况下应加入 RL?	存在明确可学习的序列决策和可测奖励，而且简单基线不足以回答问题
17. 课程学习会如何泄漏测试信息?	根据测试表现调整难度、生成器或采样规则会把测试变成开发反馈
18. 超时能否当成缺失值直接删除?	不应；它是算法可靠性和预算结果的一部分，删除会偏向成功样本
19. OpenEvolve 是否是 AlphaEvolve 官方源码?	不是；它是社区实现，需按自身版本和任务评价
20. 最终没有显著改进，还交付什么?	可信复现、成本/失败分析、适用边界、可重建数据与明确负结果

14.2 概念索引

实例 instance：问题的一次具体输入；解 solution：满足任务格式的输出；可行性 feasibility：满足约束；目标 objective：评价优劣的量；算法 algorithm：从输入到输出的规则；启发式 heuristic：利用结构快速求解的策略；配置 configuration：给定结构的参数设置；选择 selection：在已知方法间决策；设计 design：搜索算法结构或代码。

种群 population：并行维护的候选集合；父代 parent：生成新候选的来源；变异 mutation：产生局部/结构变化；交叉 crossover：组合来源信息；精英保留 elitism：保留已评价的优良候选；多样性 diversity：避免候选过度同质；适应度 fitness：搜索使用的评价，不必等同单个实例的目标值。

生成器 generator：提出程序或数据的机制；评估器 evaluator：执行并量化表现；验证器 validator：独立检查格式、可行性或正确性；程序规格 specification：程序应满足的要求；反例 counterexample：违反规格的具体输入及行为；沙箱 sandbox：限制不可信程序权限与资源的执行环境；接口合同 contract：明确的输入输出和错误处理规定。

预训练 pretraining：从大规模数据学习模型参数；微调 fine-tuning：在后续数据/目标上更新参数；上下文学习 in-context learning：通过输入上下文改变行为；策略 policy：状态到动作分布；回报 return：累计奖励；模仿学习 imitation learning：从示范行为学习；课程 curriculum：安排训练任务或采样难度；分布外 OOD：与开发分布不同的评估条件。

独立重复 independent replication：使用独立随机过程重复完整方法；消融 ablation：移除或替换机制以测试贡献；效应量 effect size：改进的大小；置信区间 confidence interval：量化估计不确定性的方法；预注册 preregistration：事先写明假设和分析规则；工件 artifact：代码、数据、配置和可重建结果；最好已知解 BKS：当前已知参考，不自动是最优证明。

15 参考文献与资源登记

编号前缀 P 为论文与研究规范，V 为视频、课程及配套材料，C 为源码与工具，A 为误引核查证据。共 82 条记录；包含相同报告的不同入口及 7 条误引证据，不等于 82 门课程或 82 个独立推荐。核验截止日期统一为 2026 年 9 月 19 日。原始指南为用户提供的非公开学习材料，本册不附其原文件。

15.1 论文与研究规范

P01 ·Liu et al. A Systematic Survey on Large Language Models for Algorithm Design. arXiv:2410.14716v5, 2026-01-02.
入口：https://arxiv.org/abs/2410.14716v5
核验：一手页面已核验。按本次可读的 v5 引用；角色分类为 optimizers / predictors / extractors / designers。

P02 ·Liu et al. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. ICML 2024.
入口：https://arxiv.org/abs/2401.02051
核验：一手页面已核验。方法正文及第 3、4 页截图核验；原论文任务为在线装箱、TSP、流水车间调度。

P03 ·Romera-Paredes et al. Mathematical discoveries from program search with large language models. Nature 625, 468–475 (2024). DOI:10.1038/s41586-023-06924-6.
入口：https://deepmind.google/blog/funsearch-making-new-discoveries-in-mathematical-sciences-using-large-language-models/
核验：作者机构、官方代码与出版社检索元数据已核验；出版社直接抓取受限。FunSearch；在线发表于 2023 年，卷期为 2024 年。

P04 ·Ye et al. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. NeurIPS 2024.
入口：https://proceedings.neurips.cc/paper_files/paper/2024/hash/4ced59d480e07d290b6f29fc8798f195-Abstract-Conference.html
核验：一手页面已核验。

P05 ·Liu et al. LLM4AD: A Platform for Algorithm Design with Large Language Model. arXiv:2412.17287v2, 2026-02-26.
入口：https://arxiv.org/abs/2412.17287v2
核验：一手页面已核验。首版 2024-12-23；平台论文不等于 EoH 原论文。

P06 ·Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025.
入口：https://arxiv.org/abs/2506.13131
核验：一手页面已核验。白皮书；不把作者结果写成本文复现结果。

P07 ·Liu et al. EoH-S: Evolution of Heuristic Set Using LLMs for Automated Heuristic Design. AAAI 40(43):37090–37098, 2026.
入口：https://ojs.aaai.org/index.php/AAAI/article/view/41038
核验：一手页面已核验。DOI:10.1609/aaai.v40i43.41038；预印本 2508.03082。

P08 ·Surina et al. Algorithm Discovery With LLMs: Evolutionary Search Meets Reinforcement Learning. arXiv:2504.05108, 2025.
入口：https://arxiv.org/abs/2504.05108
核验：一手页面已核验。FunTune；搜索反馈用于 RL 微调生成器，不能称 RL+ 进化全新。

P09 ·Yang et al. HeurAgenix: Leveraging LLMs for Solving Complex Combinatorial Optimization Challenges. arXiv:2506.15196, 2025.
入口：https://arxiv.org/abs/2506.15196
核验：一手页面已核验。本手册使用其启发式生成与选择机制进行先例核查；未运行上游实验。

P10 ·Ke et al. Game-Theoretic Co-Evolution for LLM-Based Heuristic Discovery. arXiv:2601.22896, 2026.
入口：https://arxiv.org/abs/2601.22896
核验：一手页面已核验。预印本；求解器/实例生成器共同演化，影响课程学习的创新判断。

P11 ·Lai et al. ES-AHD: An Evolution Strategy Framework for Automatic Heuristic Design. arXiv:2609.00023, 2026.
入口：https://arxiv.org/abs/2609.00023
核验：一手页面已核验。近期预印本，身份和摘要核验；未独立复现，不列为已证明的最强基线。

P12 ·Li et al. Competition-Level Code Generation with AlphaCode. Science, 2022; arXiv:2203.07814.
入口：https://arxiv.org/abs/2203.07814
核验：一手页面已核验。

P13 ·Nijkamp et al. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. ICLR 2023; arXiv:2203.13474.
入口：https://arxiv.org/abs/2203.13474
核验：一手页面已核验。

P14 ·Liu, Simonyan, Yang. DARTS: Differentiable Architecture Search. ICLR 2019; arXiv:1806.09055.
入口：https://arxiv.org/abs/1806.09055
核验：一手页面已核验。

P15 ·Elsken, Metzen, Hutter. Neural Architecture Search: A Survey. JMLR 20(55), 2019; arXiv:1808.05377.
入口：https://arxiv.org/abs/1808.05377
核验：一手页面已核验。

P16 ·Jiang et al. A Survey on Large Language Models for Code Generation. arXiv:2406.00515, 2024.
入口：https://arxiv.org/abs/2406.00515
核验：一手页面已核验。

P17 ·Austin et al. Program Synthesis with Large Language Models. arXiv:2108.07732, 2021.
入口：https://arxiv.org/abs/2108.07732
核验：一手页面已核验。研究论文，不冒充原指南所写的综述。

P18 ·Vaswani et al. Attention Is All You Need. NeurIPS 2017; arXiv: 1706.03762.
入口：https://arxiv.org/abs/1706.03762
核验：一手页面已核验。

P19 ·Rozière et al. Code Llama: Open Foundation Models for Code. arXiv: 2308.12950, 2023.
入口：https://arxiv.org/abs/2308.12950
核验：一手页面已核验。

P20 ·Li et al. StarCoder: may the source be with you! arXiv:2305.06161, 2023.
入口：https://arxiv.org/abs/2305.06161
核验：一手页面已核验。

P21 ·Luo et al. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. arXiv:2306.08568, 2023.
入口：https://arxiv.org/abs/2306.08568
核验：一手页面已核验。

P22 ·Pineau et al. Improving Reproducibility in Machine Learning Research. JMLR 22(164):1–20, 2021.
入口：https://jmlr.org/papers/v22/20-303.html
核验：一手页面已核验。

P23 ·Demšar. Statistical Comparisons of Classifiers over Multiple Data Sets. JMLR 7:1–30, 2006.
入口：https://jmlr.org/papers/v7/demsar06a.html
核验：一手页面已核验。统计思想需按实际独立实验单位使用，不能机械套用分类器检验。

P24 ·Li, Parsert, Polgreen. Guiding Enumerative Program Synthesis with Large Language Models. arXiv:2403.03997, 2024.
入口：https://arxiv.org/abs/2403.03997
核验：一手页面已核验。用于理解 LLM 与形式化枚举搜索的互补，不是必须复现项目。

P25 ·Fawzi et al. Discovering faster matrix multiplication algorithms with reinforcement learning. Nature 610, 47–53 (2022). DOI:10.1038/s41586-022-05172-4.
入口：https://deepmind.google/blog/discovering-novel-algorithms-with-alphatensor/
核验：作者机构原始介绍已核验。AlphaTensor；用于与 FunSearch 区分。

15.2 视频、课程与讲义

V01 ·MIT 6.006 Introduction to Algorithms, Spring 2020.
入口：https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/
核验：一手页面已核验。课程主页含 Video Lectures、讲义和习题；英文。

V02 ·MIT The Missing Semester of Your CS Education, 2026 / 2020 archives.
入口：https://missing.csail.mit.edu/
核验：一手页面已核验。Shell、Git、调试、性能分析；英文，站点列社区中文译本。

V03 ·李沐等：动手学深度学习在线课程，2021。
入口：https://courses.d2l.ai/zh-v2/
核验：一手页面已核验。中文；作者课程页链接 B 站回放；单条 B 站播放器本次未验证。

V04 ·动手学深度学习中文版。
入口：https://zh.d2l.ai/
核验：一手页面已核验。教材及可执行代码，不单独计为视频课程。

V05 ·李宏毅：Machine Learning 2025 Spring。
入口：https://speech.ee.ntu.edu.tw/~hylee/ml/2025-spring.php
核验：一手页面已核验。中文；公开讲座链接可核验，部分校内作业平台不对外开放。

V06 ·李宏毅：第三讲，语言模型内部运作机制剖析。
入口：https://www.youtube.com/watch?v=Xnil63UDW2o
核验：一手页面已核验。由 V05 官方课程页链接；不把这节替代完整 Transformer 数学教程。

V07 ·李宏毅：第五讲，预训练—对齐。
入口：https://www.youtube.com/watch?v=Ozos6M1JtIE
核验：一手页面已核验。由 V05 链接；中文。

V08 ·李宏毅：第六讲，后训练与遗忘问题。
入口：https://www.youtube.com/watch?v=Z6b5-77EfGk
核验：一手页面已核验。由 V05 链接；中文。

V09 ·李宏毅：第七讲，推理能力与 DeepSeek-R1。
入口：https://www.youtube.com/watch?v=bJFtcwLSNxI
核验：一手页面已核验。由 V05 链接；中文，课程内容为 2025 年。

V10 ·Andrej Karpathy: Neural Networks: Zero to Hero.
入口：https://karpathy.ai/zero-to-hero.html
核验：一手页面已核验。micrograd、makemore、GPT；英文，作者视频与代码。

V11 ·Stanford CS336: Language Modeling from Scratch, Spring 2025.
入口：https://cs336.stanford.edu/spring2025/
核验：一手页面已核验。选修分词、Transformer、实验；不要求完成大模型训练。

V12 ·Hugging Face LLM Course，中文页面。
入口：https://huggingface.co/learn/llm-course/zh-CN/chapter1/1
核验：一手页面已核验。中文讲义不代表所有视频为中文；接口实作参考。

V13 ·David Silver: Reinforcement Learning course.
入口：https://www.davidsilver.uk/teaching/
核验：一手页面已核验。官方教学页重定向到作者 WordPress；MDP、策略梯度。

V14 ·Berkeley Deep Reinforcement Learning course (CS285).
入口：https://rail.eecs.berkeley.edu/deeprlcourse/
核验：一手页面已核验。从主页选择已结束学期；优先模仿学习与策略梯度。

V15 ·3Blue1Brown: Linear Algebra.
入口：https://www.3blue1brown.com/topics/linear-algebra
核验：一手页面已核验。英文可视化直觉，不能替代矩阵形状推导。

V16 ·Harvard STAT 110.
入口：https://stat110.hsites.harvard.edu/
核验：一手页面已核验。概率论；选随机变量、条件概率、期望、极限定理。

V17 ·Boyd & Vandenberghe: Convex Optimization，作者资源页。
入口：https://web.stanford.edu/~boyd/cvxbook/
核验：一手页面已核验。选约束与对偶；这是教材入口，不虚称完整免费视频。

V18 ·Armando Solar-Lezama: Program Synthesis course.
入口：https://people.csail.mit.edu/asolar/SynthesisCourse/index.htm
核验：一手页面已核验。MIT 作者课程讲义；主要看规格、搜索、CEGIS。

V19 ·Matej Balog: FunSearch，Harvard CMSA，2024-10-28。
入口：https://cmsa.fas.harvard.edu/event/mmlworkshop_1024/
核验：一手页面已核验。主办方日程及 YouTube 播放列表已核验；不宣称逐帧观看。

V20 ·Matej Balog | FunSearch: Mathematical discoveries from program search with LLMs.
入口：https://www.youtube.com/watch?v=VJ_meFdGwE8
核验：视频索引与主办方事件交叉核验；直接播放器抓取受限。与 V19 为同一报告，不重复计为两门课。

V21 ·LLM4AD: TSP video example, Colab, CEC/EMO tutorial links.
入口：https://www.llm4ad.com/
核验：一手页面已核验。首页嵌入 TSP 演示；CEC/EMO 链接为教程幻灯片，不冒充视频。

V22 ·AutoML 2023: A Survey of Methods for Automated Algorithm Configuration.
入口：https://2023.automl.cc/program/tutorials/survey-of-methods/
核验：一手页面已核验。会议官方页面含 Video；学习参数配置与算法设计边界。

V23 ·AutoML 2023 算法配置教程，tele-TASK 录像。
入口：https://www.tele-task.de/lecture/video/10161/
核验：主办方/录像索引核验；直接页面抓取受限。索引列英文及 01:27:56；本手册不依赖播放时长排计划。

V24 ·AutoML 2023: Language Modelling for Optimization.
入口：https://2023.automl.cc/program/tutorials/language-modelling-for-optimization/
核验：一手页面已核验。Xingyou Song、Yutian Chen；概念桥接课。

V25 ·AAAI 2026 EoH-S：Video/Slides。
入口：https://ojs.aaai.org/index.php/AAAI/article/view/41038
核验：一手页面已核验。已核验页面存在 Video/Slides 按钮；未宣称媒体文件已下载。

15.3 源码、基准与文档

C01 ·FeiLiu36/LLM4AlgorithmDesign（LLM4Opt）
入口：https://github.com/FeiLiu36/LLM4AlgorithmDesign
核验：一手页面已核验。文献导航，不是运行框架。

C02 ·FeiLiu36/EoH
入口：https://github.com/FeiLiu36/EoH
核验：一手页面已核验。当前 README：LLMConfig/EoH/BaseProblem；旧 Paras/eoh.EVOL 需 v0.1；未运行上游。

C03 ·Optima-CityU/LLM4AD
入口：https://github.com/Optima-CityU/LLM4AD
核验：一手页面已核验。README blob 47e55f22d5058817a4e684339ee7b8273c74e013；不是 commit；未运行上游。

C04 ·LLM4AD documentation
入口：https://llm4ad-doc.readthedocs.io/en/latest/
核验：官方 README 提供入口。安装/方法/任务/模型接口，版本需与仓库一致。

C05 ·google-deepmind/funsearch
入口：https://github.com/google-deepmind/funsearch
核验：一手页面已核验。算法结构学习；模型、沙箱、规模须单独核对。

C06 ·ai4co/reevo
入口：https://github.com/ai4co/reevo
核验：一手页面已核验。反思进化；未运行上游。

C07 ·FeiLiu36/EoH-S
入口：https://github.com/FeiLiu36/EoH-S
核验：一手页面已核验。互补集合设计；用于先例核查和进阶比较。

C08 ·algorithmicsuperintelligence/openevolve（原 codelion 路径重定向）
入口：https://github.com/algorithmicsuperintelligence/openevolve
核验：一手页面已核验。独立社区实现，不是 DeepMind 原版 AlphaEvolve。

C09 ·XAI-liacs/LLaMEA（原 nikivanstein 路径重定向）
入口：https://github.com/XAI-liacs/LLaMEA
核验：一手页面已核验。优化器自动设计；先读 README 与评估配置。

C10 ·microsoft/HeurAgenix
入口：https://github.com/microsoft/HeurAgenix
核验：一手页面已核验。启发式生成与选择的既有工程。

C11 ·DEAP
入口：https://github.com/DEAP/deap
核验：一手页面已核验。进化操作及 fitness 生命周期练习。

C12 ·DEAP: Operators and Algorithms
入口：https://deap.readthedocs.io/en/master/tutorials/basic/part2.html
核验：一手页面已核验。clone 和 fitness 失效必须理解；不能只照抄算子。

C13 ·PyVRP/PyVRP
入口：https://github.com/PyVRP/PyVRP
核验：一手页面已核验。当前 README 描述 ILS；历史 2024 论文与最新主分支不是同一实现。

C14 ·vidalt/HGS-CVRP
入口：https://github.com/vidalt/HGS-CVRP
核验：一手页面已核验。README 指向 v1.0.0 对应论文；读取 round/seed/time 设置。

C15 ·Google OR-Tools: Vehicle Routing Problem
入口：https://developers.google.com/optimization/routing/vrp
核验：一手页面已核验。模型与回调接口；启发式解不自动构成最优性证明。

C16 ·N-Wouda/ALNS
入口：https://github.com/N-Wouda/ALNS
核验：一手页面已核验。破坏/修复/选择/接受分离；不是预先指定最终研究框架。

C17 ·ai4co/rl4co
入口：https://github.com/ai4co/rl4co
核验：一手页面已核验。构造与改进策略；README 含文档、Issues、Discussions、Slack。

C18 ·openai/human-eval
入口：https://github.com/openai/human-eval
核验：一手页面已核验。功能正确性；README 明确警告执行不可信代码需要沙箱。

C19 ·bigcode-project/bigcodebench
入口：https://github.com/bigcode-project/bigcodebench
核验：一手页面已核验。页面显示 2026-07-20 归档；功能调用与复杂指令评估，未运行上游。

C20 ·hendrycks/apps
入口：https://github.com/hendrycks/apps
核验：GitHub 连接器读取 README 已核验。APPS 数据与训练/评价入口；网页直接抓取超时，但连接器成功。

C21 ·salesforce/CodeGen
入口：https://github.com/salesforce/CodeGen
核验：一手页面已核验。官方模型及程序合成入口；不作为 2026 最新推荐。

C22 ·bigcode-project/starcoder
入口：https://github.com/bigcode-project/starcoder
核验：一手页面已核验。官方微调与推理入口；运行与许可需按实际模型版本。

C23 ·meta-llama/codellama
入口：https://github.com/meta-llama/codellama
核验：一手页面已核验。推理代码入口；未下载权重，未验证当前设备兼容性。

C24 ·google-research/nasbench
入口：https://github.com/google-research/nasbench
核验：一手页面已核验。真实仓库名为 nasbench；NAS 数据与基准，非 LLM 启发式主线。

C25 ·google-deepmind/code_contests
入口：https://github.com/google-deepmind/code_contests
核验：一手页面已核验。AlphaCode 相关公开数据/材料，不等于完整原系统发行版。

15.4 原指南误引的核查证据

A01 ·COWPEA (Candidates Optimally Weighted in Proportional Election using Approval voting)
入口：https://arxiv.org/abs/2305.08857
核验：一手页面已核验。原指南错配的真实标识符；仅用于查错，不是推荐阅读。

A02 ·Probing Early Modification of Gravity with Planck, ACT and SPT
入口：https://arxiv.org/abs/2308.12345
核验：一手页面已核验。原指南错配的真实标识符；仅用于查错，不是推荐阅读。

A03 ·Once-for-All: Train One Network and Specialize it for Efficient Deployment
入口：https://arxiv.org/abs/1908.09791
核验：一手页面已核验。原指南错配的真实标识符；仅用于查错，不是推荐阅读。

A04 ·High-quality axions in a class of chiral U(1) gauge theories
入口：https://arxiv.org/abs/2301.02345
核验：一手页面已核验。原指南错配的真实标识符；仅用于查错，不是推荐阅读。

A05 ·On Finite Analogs of Schmidt's Problem and Its Variants
入口：https://arxiv.org/abs/2205.00527
核验：一手页面已核验。原指南错配的真实标识符；仅用于查错，不是推荐阅读。

A06 ·The doubly librating Plutinos
入口：https://arxiv.org/abs/2501.12345
核验：一手页面已核验。原指南错配的真实标识符；仅用于查错，不是推荐阅读。

A07 ·原指南 LLM4AD 链接：arXiv:2502.23456
入口：https://arxiv.org/abs/2502.23456
核验：本次返回 404。修正为 P05，不据此断言一切同名资料不存在。